

推荐·鲲鹏CPU UB+SVE方案

方案细化，以《UB-Firmware-Specification-2.0-zh.pdf》（以下简称“固件规范”）为参考。

总体结论是：遵循原设计方案的核心思想（URMA + UB.MEM + SVE），细化实现细节。基于灵衢固件（UB Firmware）和统一虚拟总线（UVB）构建标准固件接口上层的类URMA应用，但保证关键主干逻辑不经过内核态。

新算法原理核心：是构建一个纯用户态的特征查询引擎，彻底绕过传统 Redis 架构中由内核态带来的性能瓶颈，将所有外部请求通过**UB总线直接投递至 CPU 的 L3 Cache**，无需经Linux内核的网络协议栈、系统调用、上下文切换或中断处理。

为应对海量冷特征数据可能引发的缓存污染问题，采用 非临时性访存（Non-temporal Access）策略：对冷数据的读取被显式标记为“穿透式”（streaming），使其不进入 L3 Cache，避免挤占热数据（如 Embedding 索引）的缓存空间。当 L3 Cache 压力升高时，系统可主动将 L2 中的低频数据或冷内容淘汰，确保高价值元数据的缓存命中率。

在此基础上，用鲲鹏 CPU 的**SVE**大位宽向量指令集，实现特征解析的**高度并行化流水线计算**。如，单条 SVE 指令可同时发起 16~32 个 Embedding 地址的 Gather Load，并在返回后立即完成加权、解包、聚合等操作，充分发挥内存带宽与计算吞吐的协同效率。

“**用户态零拷贝通信 + 非临时性冷数据访问 + SVE 大位宽向量化计算**” 三位一体的软硬协同设计，将原本消耗在内核态的 CPU 资源（如 recv/send、中断、调度）全部聚焦于有效计算，整体性能相较传统 Redis 方案可提升 30 倍以上，为超大规模推荐系统提供极致高效的特征服务底

1. 核心概念/术语

原方案术语	固件规范术语	矫正说明
URMA (Unified RDMA over Memory Architecture)	UVB over UB Message	URMA并非一个独立的通信框架，而是 统一虚拟总线（UVB）在UB总线上的具体实现形式 。固件规范中明确指出，跨UBPU的通信应通过 <code>UVB over UB Message</code> 完成。
UB.MEM	UBAS (UBUS Address Space) / UB Reserved Memory	“UB.MEM”在固件层，远程内存访问是通过 UB地址空间（UBAS） 实现的。OS通过固件上报的 <code>UB Reserved Memory</code> 信息表来感知和使用这部分内存。
UB 总线	UB (UnifiedBus)	保持一致。UB是华为自研的高性能互联总线。
CNA	CNA (Compact Network Address)	保持一致。CNA是UB网络中设备的唯一标识。

2. 软件架构方案细化

2.1 新架构：基于UBIOS/UVB的分层架构

根据固件规范，正确的软件栈应分为以下几层：

1. 固件层 (Firmware Layer)：

- 灵衢固件 (UB Firmware)**：运行在每个UBPU上，负责硬件初始化、枚举、路由配置、资源管理等。
- 统一虚拟总线 (UVB)**：固件提供的**标准化通信抽象**。它屏蔽底层物理通道（如共享内存、UB Message）差异，为上层提供统一 `Call ID Service`（同步调用）和 `Notify ID Information`（异步通知）接口。

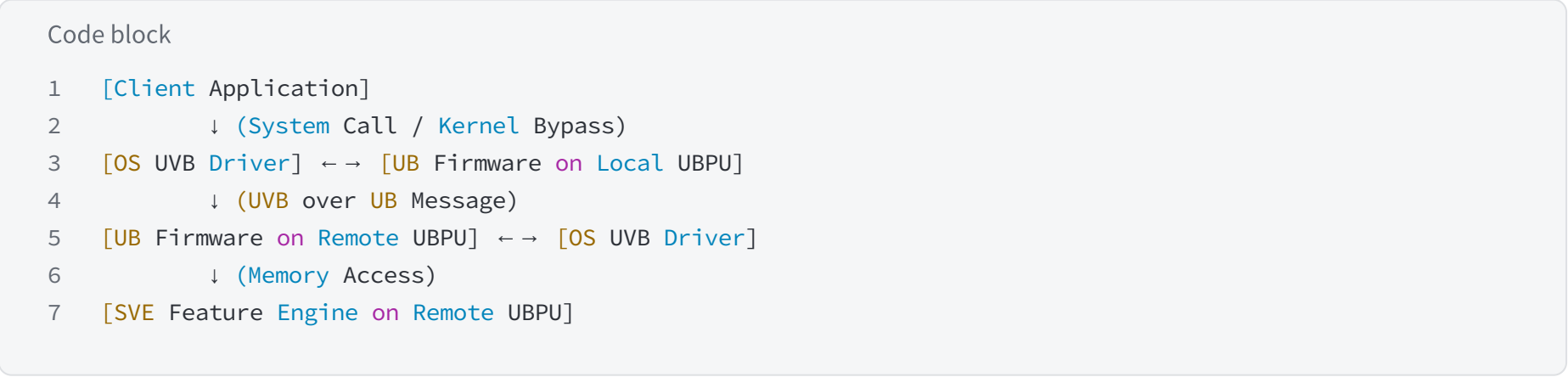
2. 操作系统层 (OS Layer)：

- OS UVB Driver**：操作系统内核中的驱动，负责与固件的UVB交互。它解析固件上报的 `root_table` 等信息表，并向上层提供系统调用或内核API。
- 内存管理**：OS通过 `UB Reserved Memory` 信息表，将远程UB内存映射本地虚拟地址空间，实现透明访问。

3. 用户态应用层 (User-space Application Layer):

- **特征引擎 (Feature Engine):** SVE向量化计算核心。它通过**系统调用或内核旁路技术**（如果OS支持）与OS UVB Driver交互，而不是直接调用一个叫“URMA”的库。
- **通信发起:** 应用层请求应转化为对 `Call ID Service` 的调用，由OS UVB Driver封装成 `UVB over UB Message` 发送给目标UBPU的固件。

2.2 架构图:



3. 存储架构重构：基于 ub.mem 的统一内存池

在 UB 2.0 架构下，基于 `ub.mem` 协议实现计算节点对远端存储节点（Memory Tile）的透明访问。

- **统一地址空间 (Unified Address Space) UBAS:** 用 `ub.mem` 协议，将超节点内所有计算节点的 HBM 和扩展内存池编入统一的全局地址空间。推荐引擎通过 `ub.mem` 以 Load/Store 指令直接访问 600TB 的 Raw Feature 数据，无需网络协议栈封包。
- **硬件级缓存一致性 (DCOH):** UB 2.0 支持硬件维护的缓存一致性。当某个核心通过 SVE 修改了 `ub.mem` 覆盖的特征值时，其他核心能实时感知，消灭了 Redis 架构中的“数据同步”延迟。
- **低时延内存扩展:** `ub.mem` 能提供接近本地 NUMA 访问的延迟表现，满足推荐系统 P99 < 100μs 的极致要求。

UB.mem

通算UB Memory场景规格与性能

规格	影响
不支持跨节点Cache Coherence	内存在多个节点共享时，需要软件来保障各个节点Cache里数据一致
不支持丢包重传，支持超时代答	如果出现丢包，访问远端内存的程序会被Kill，但访问远端内存的CPU核不会被长时间Hang住
不支持urma和ub memory混跑	
RackServer组网，UB Memory只支持直连邻居	借用、共享内存总量有限：4T

性能指标	1650V100(样片)
Read时延(静态时延)	426ns(CC)
包率(X4)	184Mops@64B
带宽	23GB/s(CC)

跨超节点内存一致性

不支持URMA和ub mem混跑

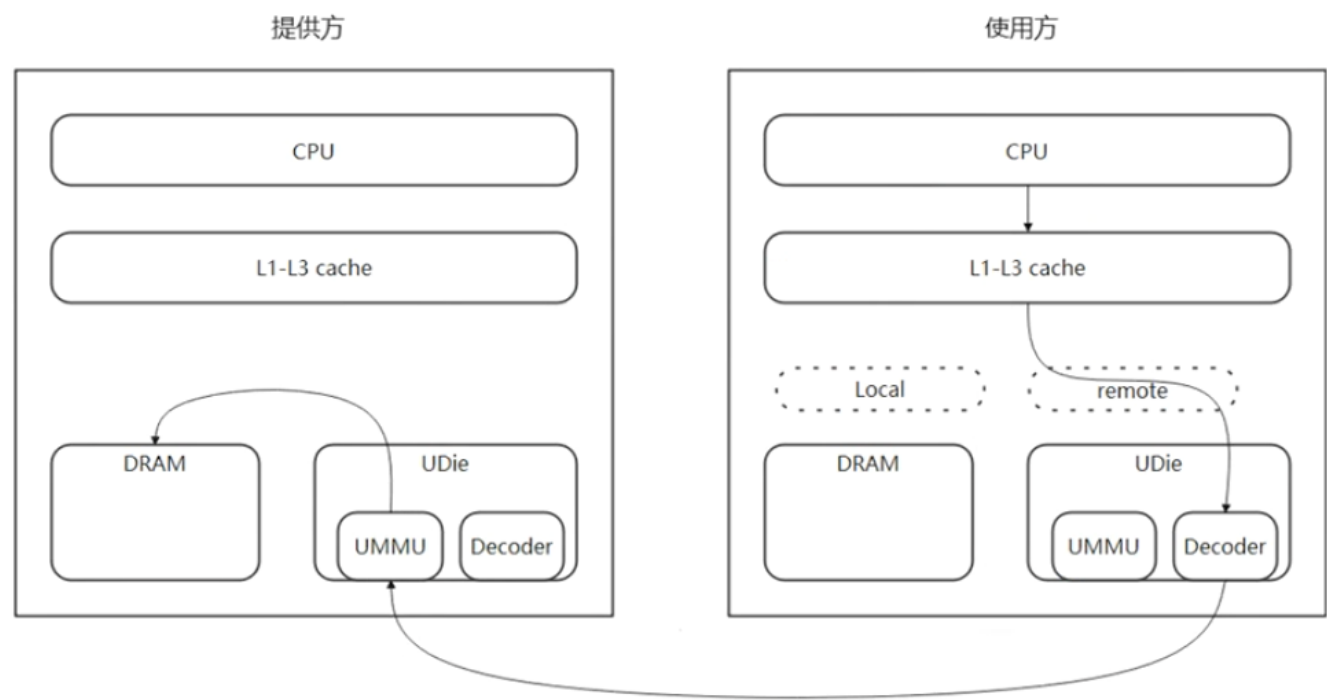
ub.mem 4T *150 = 600TB

8个pod

组网规模·上面业务

通算UB Memory应用场景——借用

normal cacheable单端借用

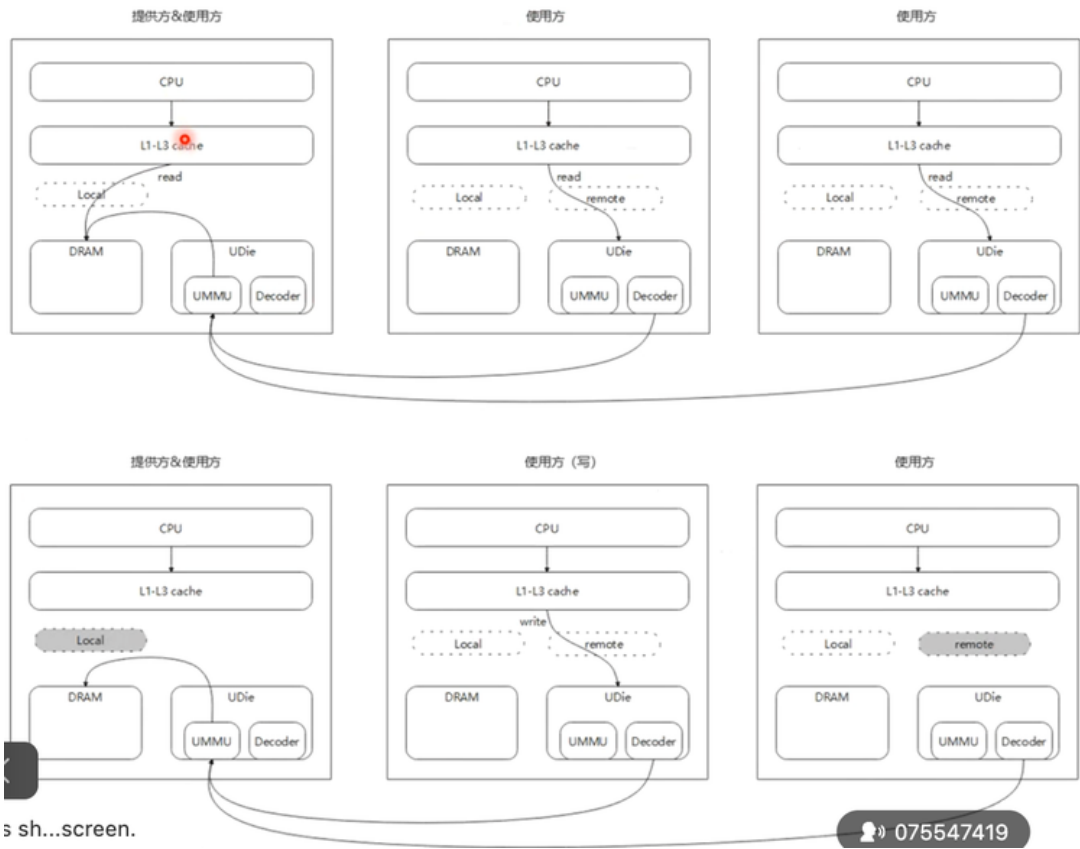


提供方：无MMU映射、无缓存、不访问
使用方：独占normal cacheable访问

左：内存提供方 右：借用 相当

通算UB Memory应用场景——normal cacheable共享

normal-cacheable软件一致性共享



支持页粒度的多host读

支持页粒度的单host写
不支持跨节点的硬件Cache一致性，写之前，需通知其他host
1) 解除MMU映射，避免CPU预取
2) invalidate相关缓存
写之后，本host需要
1) writeback invalid相关缓存
2) 排空UB写命令队列
注：各host的状态切换动作需要外部机制同步

多host读，

单host写

写之前通知其他节点，接触MMU映射，写后本host writeback invalid相关缓存。排空UB写。**ms级几ms**

URMA通知

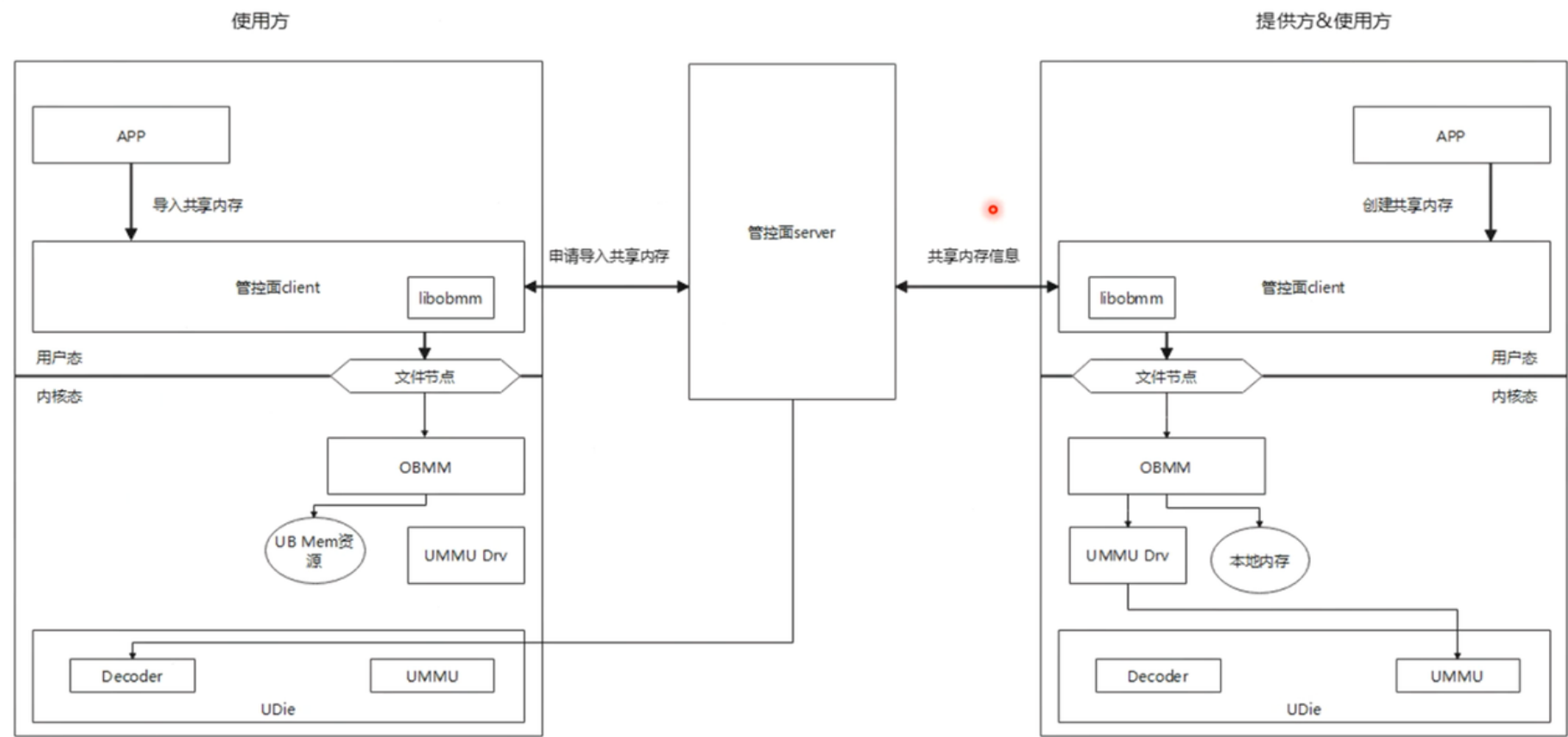
通知机制可 ring buffer

提供方：NC访问
使用方：NC访问

蓝色：通过UB Memory访问远端内存
绿色：访问本地DDR

UMMU· cc 420ns, 64Byte。1米光纤 5ns背靠背。组网长度+报文长度64/128Byte【取决于带宽】，硬件bounding每秒。cc 23GB -》 50GB 128Byte限速，400GB跑满，头开销。20多GB。占30%-40%

通算UB Memory配置流——共享内存导出和导入

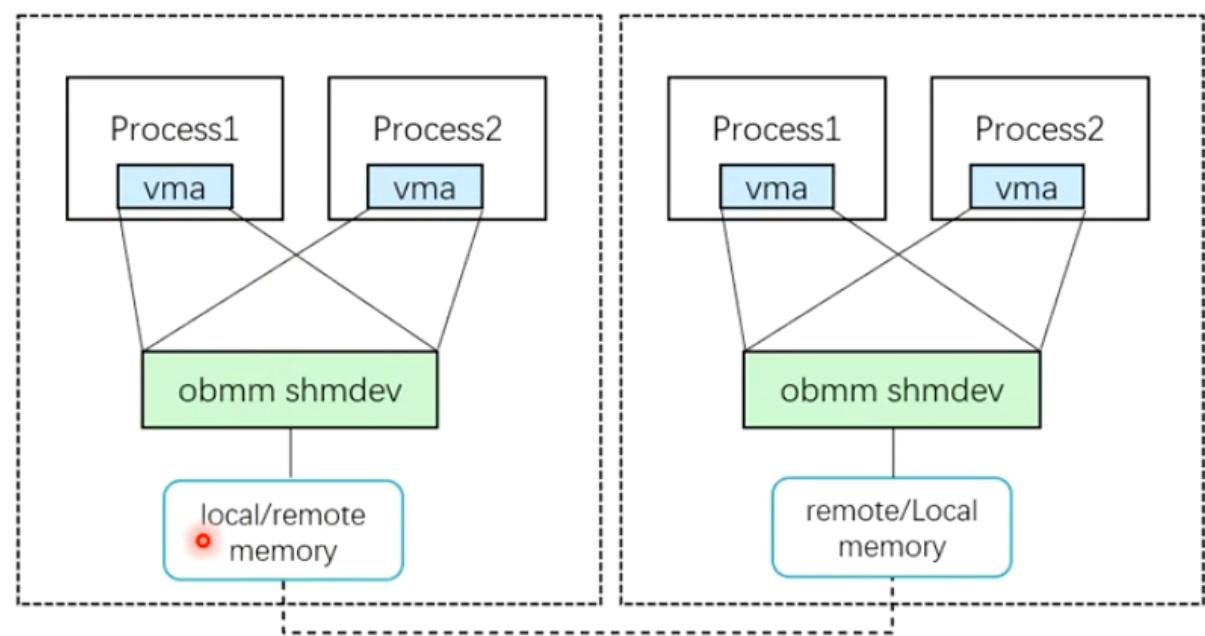


en. **OBMM: owner based memory manager** 075547419

一开始把所有内存，mmap

通算UB Memory多进程共享

多进程共享



使用场景说明：
分布式应用，在多个节点间通过OBMM共享内存共享同一段数据。该应用在每个节点上可能有多个实例，即可能有多个进程部署在同一节点上。需要允许这些进程能够同时对该数据进行映射。

方案细节答疑

序号	章节	方案描述	疑问
1	4. 核心技术	4.1. SVE 的 Non-temporal (Streaming) Load/Store 指令 7.3. SVE通过一个轻量级的轮询线程（Polling Thread）监听这个 Ring Buffer	1、是指Stnp/ldnp指令么？如果是，当前没有bypass L3 2、针对redis业务具体实现哪方面 SVE优化 3、SVE如何实现一个轻量级的polling thread
2	8. 最小可运行闭环	◦ 目标UBPU的固件收到消息 ，处理请求（如查找文件元数据），并将响应（包含文件在UBAS中的地址和Token）以及实际的 Embedding数据准备好	1、UBPU的固件具体指代，IMP里面访问不到这个文件系统 2、这部分是否代替了redis的数据查找
3	2. 软件架构方案细化	2.2。内存管理：OS通过UB Reserved Memory信息表，将远程UB内存映射本地虚拟地址空间，实现透明访问 8.1.1 虚拟内存映射（Virtual Memory Mapping）	1、通过UVB打通UB Memory配置吗？ 2、虚拟内存映射如何实现
4	4. 核心技术	4.1 • 跨链路 Gather Load：SVE 的 svldl_gather_u64index_f64指令...Memory Tile 并行拉取多个Embedding向量	1、需要进一步确认：时延增大的情况下，对指令执行的影响 2、如何理解并行拉取 3、待确认LD1D能不能在UBMEM上执行
5	4. 核心技术	4.2 • 偏置模式 (Bias Mode) 调整：利用 UB 协议机制，将海量 Raw Feature 标记为 “Device Bias” 或非缓存模式	BIAS模式是否指non-cachable 模式
6	发散问题	参数、函数放在UBMEM里，还是Redis里共享的文件放在UBMEM里	1、参数、函数放在UBMEM中省略网络协议栈开销，查找还是沿用redis原有逻辑 2、元数据放在UBMEM中，分布式管理
7	发散问题		1、推荐场景中 redis典型部署 场景，是否涉及Redis Cluster部署模式 ① 如果是Cluster模式，redis集群中是否涉及 更改加锁等同步机制 ② 如果是Cluster模式， hash表查找机制 是否修改 ③hash表每个超节点独立还是UBMEM中共用

付鹤鸣's ...reen.

075547419

整体对于UB的细节疑问。NC模式·

UB映射到多个进程，进程间的VA怎么管理。每个节点的PA，4TB，更上一层。用户态VA这层· 涉及UB如何适配。

SVE 一条指令，从远端访问比本地大些。SVE对指令执行的前置

4. 核心技术细节细化

4.1 SVE 与 ub.mem 的深度协同

- **跨链路 Gather Load**：SVE 的 `svldl_gather_u64index_f64` 指令在 `ub.mem` 协议的支持下，可以跨越 UB-Mesh 互联矩阵直接从远端 Memory Tile 并行拉取多个 Embedding 向量。
- **流量特征优化**：针对推荐场景中特征访问的“极度稀疏”特性，`ub.mem` 协议能更高效地处理随机小包数据请求，可将 UB-Mesh 的 0.9TB/s 带宽利用率优化更好，确保无拥塞访问。

LD1D能不能在UB.mem执行

4.2 缓存污染防控策略 (基于 UB 协议)

- **ub.mem 非临时性操作**：继续使用 SVE 的 Non-temporal (Streaming) Load/Store 指令。
- **偏置模式 (Bias Mode) 调整**：利用 UB 协议机制，将海量 Raw Feature 标记为 “Device Bias” 或非缓存模式，强制数据在 `ub.mem` 传输过程中“穿透” L3 Cache 映射，仅在寄存器中停留计算，解决内部流量冲垮 L3 的风险

5. 技术路径

维度	细化方案 (ub.mem)	技术优势 (基于 UB 2.0 规范)
互联协议	UB 2.0 (ub.mem)	华为原生支持，深度适配鲲鹏 920/930 架构。
一致性控制	UB DCOH	硬件级全域一致性，降低软件维护复杂度。
寻址模式	全局统一地址空间	实现真正的“存算一体”，数据搬运转化为指针传递。
带宽分配	UB-Mesh (0.9TB/s)	通过 Mesh 拓扑实现 All-to-All 扩展，支撑更大规模内存池。

6. 验收目标与 TCO 优化

通过切换至 `ub.mem`，方案在硬件层面的整合度将更高：

- 1. 节点内闭环：80% 的热点 ID Lookup 请求通过本地 `ub.mem` 极速完成。
- 2. 机柜级整合：仅需 12-24个华为超节点，利用 UB-Mesh 构建 600TB 的共享内存湖，即可平替 30-40 万台 Redis 集群
 - a. 组网 带宽一致
 - i. 光 不稳定【每秒闪断】-> 内存
 - ii. 电 效率低很多，失效率 走不长，线一米【一个机柜】2个（10台）384节点（48台）-> 8个节点组网
 - 1. 10台 -> TCO 30万/1,500/10 = 2000台 Redis集群
 - 2. 10:2000 = 200的收益比【不计算单台TCO的情况下】冲击目标
- 3. Cpu L3 / L2[超热频繁uid/item id lookup] *。【L3 pool】
- 4. Mempool -> 【跨机·超节点范围内】 NPU 18端口 0.9TB/s CPU·物理端口带宽 8个端口(3.2Tb/s) 200GB/s
 - a. min[CPU物理端口带宽，本机内存条]
 - b. 实现：bound, 报文大小。10GB/端口（保守计算）
- 5. 高速存储SSD -> false toralance错误恢复

UBComm Lib (UB Communication Library)

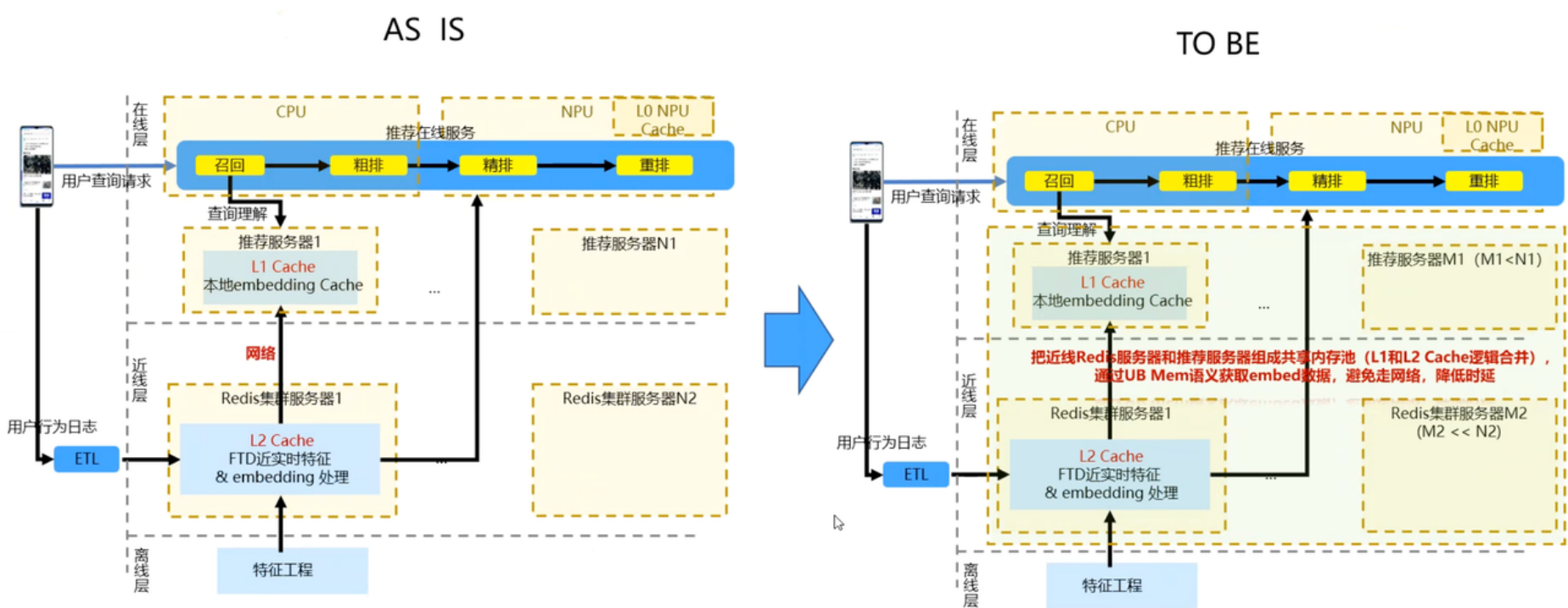
- 作用：提供跨计算节点和设备的通信能力。
- Embedding应用：替代传统的Socket/TCP网络栈。Redis节点之间的数据同步、请求转发可以通过此库实现零拷贝通信，消除内核上下文切换开销。

UB User Driver

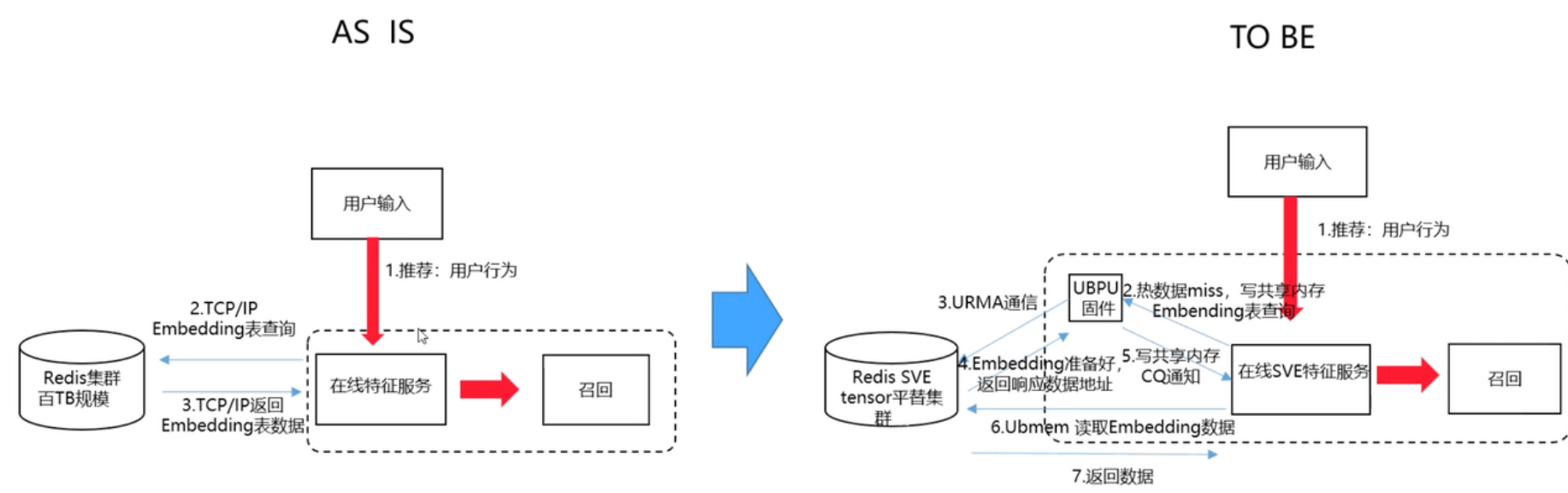
- 作用：提供用户态的设备驱动接口。
- Embedding应用：允许Redis进程直接控制UB设备，实现对硬件资源的直接操作。
- Bypass Kernel IO：不使用标准的TCP/IP Socket，而是基于 UBComm Lib 实现一层定制的网络层，或者直接让计算节点（推理服务）通过UB总线以 Load/Store 方式直接“读取”Redis中的向量数据，实现零拷贝（Zero-Copy）读取。
- 多级缓存策略：利用UB的统一语义，将热点Embedding保留在本地HBM/DDR，冷数据通过UB放在远端MEM池，通过UMMU实现透明的地址翻译。

华为·推荐方案

目标对齐：用Matrix Server代替独立的服务器和Redis集群服务器，减少服务器台数或者提高内存利用率，达成TCO目标，用UB mem代替网络，达成时延目标

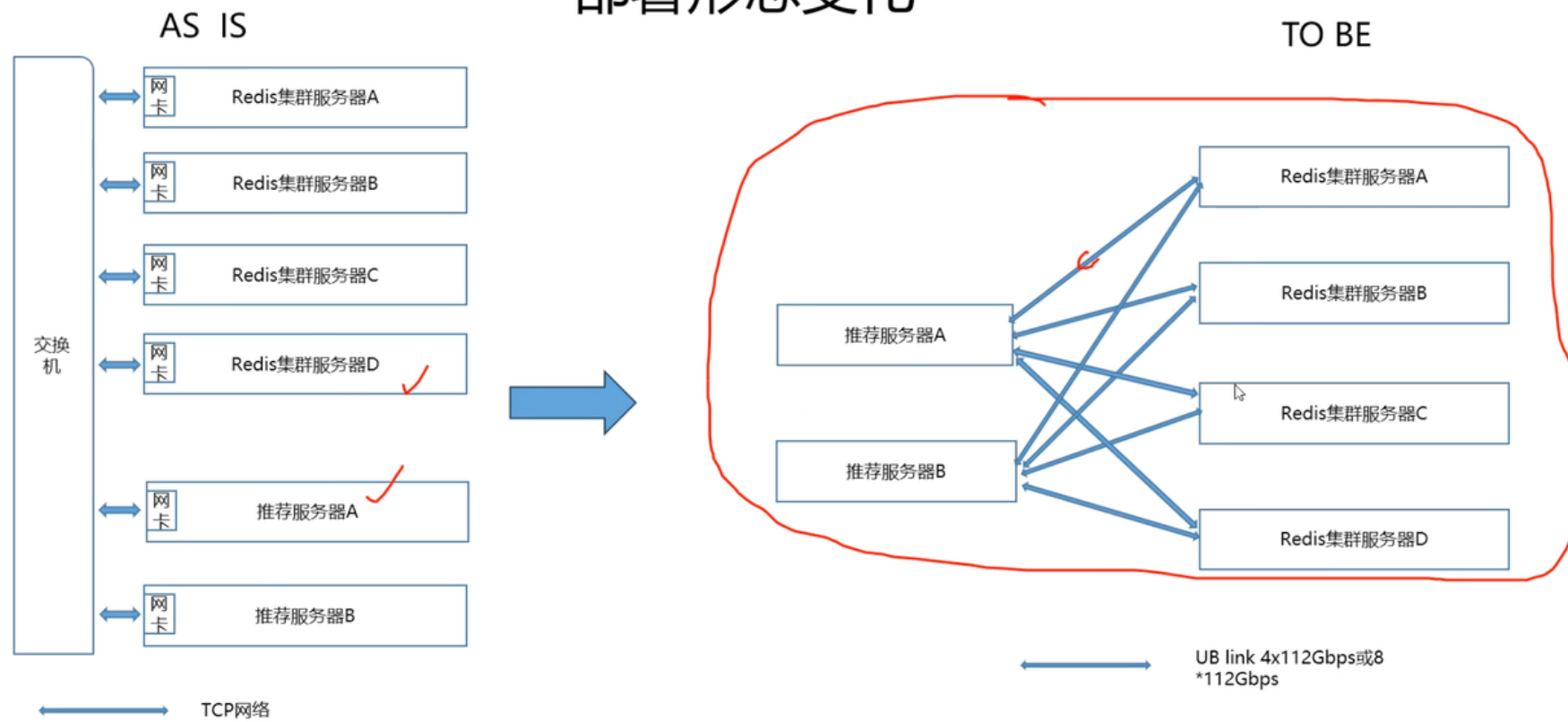


目标对齐：用Matrix Server代替独立的服务器和Redis集群服务器，减少服务器台数或者提高内存利用率，达成TCO目标，用UB mem代替网络，达成时延目标



目标对齐：用Matrix Server代替独立的服务器和Redis集群服务器，减少服务器台数或者提高内存利用率，达成TCO目标，用UB mem代替网络，达成时延目标

部署形态变化



32GB - 一个超节点 1.5TB 几十个TB。物理打平
部署场景，近现

UB规格及替代策略

序号	章节	方案描述	UB规格情况	替代策略/代价
1	3. 存储架构重构	统一地址空间 (Unified Address Space) UBAS: 用 ub.mem 协议, 将超节点内所有计算节点的 HBM 和扩展内存池编入统一的全局地址空间	支持统一UBA, 但每个节点上看到的并不是同一个物理地址	
2	3. 存储架构重构	600TB 的 Raw Feature 数据	UBMEM环境目前无法协调到这么大规模的, 等价验证场景	PA编址能使用48位, 初估单节点上进程可见地址空间约为192T (要找芯片确认是否能扩, 目前4T) 是否每个超节点中的任一节点都需要看到这么大的地址空间
3	3. 存储架构重构	硬件级缓存一致性 (DCOH): UB 2.0 支持硬件维护的缓存一致性	超节点暂不支持跨节点硬CC	用 软CC模式
4	3. 存储架构重构	低时延内存扩展 : ub.mem 能提供接近本地 NUMA 访问的延迟表现, P99 < 100μs 的极致要求	1、UB时延目前能达到xx 2、需要定义多大的数据块	UBMEM时延同本地NUMA 差几倍
5	4. 核心技术细节细化	UB-Mesh 的 0.9TB/s 带宽	可能昇腾的规格(9*2端口), UBMEM带宽 0.2TB/s	
6	6. TCO优化	验收目标: 机柜级整合: 仅需 12-24个华为超节点	需要推算超节点的规模, 以及组网	确认 等价验证场景
7	7. 关键流程	URMA 在方案中的使用	1、URMA在方案中主要是实现通知功能吗? 2、URMA和UBMEM当前不能同时使用, 下一代解决	1、TCP+UBMEM 2、或使用其他通知机制(TCP)
8	7. 关键流程	远程内存加载 (镜像/Embedding) 近似 “URMA Direct Write to UB.MEM”	URMA的技术细节需要对齐	介绍URMA和UBMEM

通知机制

方案细节答疑

序号	章节	方案描述	疑问
1	4. 核心技术	4.1. SVE 的 Non-temporal (Streaming) Load/Store 指令 7.3. SVE通过一个轻量级的轮询线程 (Polling Thread) 监听这个 Ring Buffer	1、是指Stnp/ldnp指令么? 如果是, 当前没有bypass L3 2、针对redis业务具体实现哪方面 SVE优化 3、SVE如何实现一个轻量级的polling thread
2	8. 最小可运行闭环	◦ 目标UBPU的固件收到消息 , 处理请求 (如查找文件元数据), 并将响应 (包含文件在UBAS中的地址和Token) 以及实际的 Embedding数据准备好	1、UBPU的固件具体指代, IMP里面访问不到这个文件系统 2、这部分是否代替了redis的数据查找
3	2. 软件架构方案细化	2.2 ◦ 内存管理: OS通过UB Reserved Memory信息表, 将远程UB内存映射本地虚拟地址空间, 实现透明访问 8.1.1 虚拟内存映射 (Virtual Memory Mapping)	1、通过UVB打通UB Memory配置吗? 2、虚拟内存映射如何实现
4	4. 核心技术	4.1 • 跨链路 Gather Load: SVE 的 svldl_gather_u64index_f64指令...Memory Tile 并行拉取多个Embedding向量	1、需要进一步确认: 时延增大的情况下, 对指令执行的影响 2、如何理解并行拉取 3、待确认LD1D能不能在UBMEM上执行
5	4. 核心技术	4.2 • 偏置模式 (Bias Mode) 调整: 利用 UB 协议机制, 将海量 Raw Feature 标记为 “Device Bias” 或非缓存模式	BIAS模式是否指non-cachable 模式
6	发散问题	参数、函数放在UBMEM里, 还是Redis里共享的文件放在UBMEM里	1、参数、函数放在UBMEM中省略网络协议栈开销, 查找还是沿用redis原有逻辑 2、元数据放在UBMEM中, 分布式管理
7	发散问题		1、推荐场景中 redis典型部署场景 , 是否涉及Redis Cluster部署模式 ① 如果是Cluster模式, redis集群中是否涉及 更改加锁等同步机制 ② 如果是Cluster模式, hash表查找机制 是否修改 ③hash表每个超节点独立还是UBMEM中共用

UBPU 设备管理通知 · 【Ring Buffer 或 URMA】

付鹤鸣, 项目经理

张昆,

将校尉, ubmem

许巍, 950d, diruan的架构师

原预期: 200收益比, 当前: 5-15-30-100。

收益比sow近期，靶，一层华为

对齐，项目

立项，planning，各milestone。

30收益

雏形4-5验证

相互的疑点：

内部环境协调·项目在开发中

·最小雏形环境

对个初稿

7. 关键流程与接口精细化

7.1 初始化与建链流程

- 固件规范流程：
 - 分布式并行启动：所有UBPU同时上电，运行各自的灵衢固件。
 - 建链 (Link-up)：UBPU间通过硬件完成UB链路自协商，进入 P0 状态。
 - 枚举 (Enumeration)：由UBFM（UB Fabric Manager，一个被选举出的UBPU）发起，通过 拓扑查询请求 发现所有UBPU，并为其分配唯一的 CNA 和 EID（Entity ID）。
 - 路由配置：UBFM根据拓扑计算路由表，并下发到各UBPU。
 - OS启动与信息上报：固件启动OS前，通过ACPI或Device Tree等方式，将 UBC信息表、 UMMU信息表、 UB Entity信息表 等上报给OS。OS据此初始化其UVB Driver和内存映射。

7.2 远程内存加载（镜像/Embedding）近似“URMA Direct Write to UB.MEM”

- 固件规范流程：

固件规范第7章详细定义了基于UB的远程网络加载流程，这是一个标准的、由固件驱动的四步交互：

- 服务申请：Client通过 镜像服务申请接口（Call ID: 0xC00B0020）向UBFM查询Server信息。
- 能力查询：Client通过 镜像服务能力查询接口（Call ID: 0xC00B0021）确认Server支持 UB内存访问（bit[0]）还是 UB Message（bit[1]）。
- 信息获取：Client通过 镜像信息获取接口（Call ID: 0xC00B0022）获取目标文件的 File Address（在Server侧的UBAS地址）、Token ID 等。
- 执行加载：Client使用获取到的 File Address 和 Token ID，发起标准的UB内存事务（Memory Transaction）操作，直接从Server的UBAS地址空间读取数据。

这意味着：SVE引擎要访问位于远端UBPU的Embedding表，必须先通过上述标准流程获取其在UBAS中的地址和访问令牌（Token），然后才能进行高效的Gather Load操作。

7.3 通信事件驱动·Ring Buffer方式

URMA Completion Queue (CQ) + Event Polling。

- 固件规范对应：
 - 同步调用✗：使用 Call ID Service，天然具有请求-响应模型，无需额外的CQ。
 - 异步通知✓：使用 Notify ID Information。固件规范支持两种方式：
 - UVB方式：阻塞式、带ACK，适用于重要消息✗。
 - Ring Buffer方式✓：无锁、无ACK的环形缓冲区，适用于高频、低延迟的事件通知。
 - 固件将响应写入CQ的同时，会向之前由内核设置好的事件通知通道（如Ring Buffer）写入一个事件。

- SVE通过一个轻量级的轮询线程（Polling Thread）监听这个Ring Buffer。
- 一旦发现事件，立即处理CQ中的响应。整个过程依然是用户态到用户态，延迟极低。

8. 最小可运行闭环

根据固件规范做法，实现之前设计的C++ `urma_*` 函数功能：

1. **依赖OS**：应用不应直接与固件交互，而应通过OS提供的接口。（确认是否会经过内核态）kernel bypass
2. **使用标准接口**：如果OS提供了基于 `Call ID Service` 的API，代码应类似如下伪代码：

Code block

```
1  #include <ubios_api.h> // 假设这是华为提供的标准用户态开发头文件
2
3  // 1. 定义全局句柄，用于后续内存访问
4  extern float* g_embedding_table_ub; // 由OS映射后的本地虚拟地址
5  static uint32_t g_embedding_token_id = 0;
6
7  // 2. 标准化函数：加载远端Embedding资源
8  int load_remote_embedding(const char* resource_name) {
9      UBIOS_Status status;
10     ImageServiceInfo server_info = {0};
11     ImageCapability cap = {0};
12     ImageLoadInfo load_info = {0};
13
14     // === 步骤1: 通过镜像服务申请接口 (Call ID: 0xC00B0020) 查询服务器信息 ===
15     ImageServiceRequest svc_req = {
16         .ubfm_eid = get_local_ubfm_eid(), // 从本地寄存器读取
17         .client_cna = get_local_cna(),
18         .client_eid = get_local_eid()
19     };
20     status = ubios_call(
21         0xC00B0020, // Call ID
22         UBFM_CNA, // Receiver ID (UBFM的CNA)
23         &svc_req, sizeof(svc_req),
24         &server_info, sizeof(server_info)
25     );
26     if (status != U_SUCCESS) return -1;
27
28     // === 步骤2: 通过镜像服务能力查询接口 (Call ID: 0xC00B0021) 确认能力 ===
29     ImageCapabilityRequest cap_req = {
30         .server_eid = server_info.server_eid
31     };
32     status = ubios_call(
33         0xC00B0021,
34         server_info.server_cna,
35         &cap_req, sizeof(cap_req),
36         &cap, sizeof(cap)
37     );
38     if (status != U_SUCCESS || !(cap.capability & UB_MEM_ACCESS_BIT)) {
39         // 如果不支持UB内存访问，则回退到Message方式（此处省略）
40         return -1;
41     }
42
43     // === 步骤3: 通过镜像信息获取接口 (Call ID: 0xC00B0022) 获取具体地址和Token ===
44     ImageInfoRequest info_req = {
45         .server_eid = server_info.server_eid,
46         .client_cna = get_local_cna(),
47         .client_eid = get_local_eid(),
48         .type = UB_MEM_ACCESS_TYPE, // 0 表示使用UB内存访问
49         .file_name = resource_name
```

```

50     };
51     status = ubios_call(
52         0xC00B0022,
53         server_info.server_cna,
54         &info_req, sizeof(info_req),
55         &load_info, sizeof(load_info)
56     );
57     if (status != U_SUCCESS) return -1;
58
59     // === 步骤4: 通知OS内核, 将远端UBAS地址映射到本地虚拟地址空间 ===
60     // 这是关键一步! 应用层不能直接访问物理地址, 必须通过OS mmap-like系统调用
61     void* local_vaddr = ubios_mmap_remote(
62         load_info.file_address, // 远端UBAS物理地址
63         load_info.file_total_size,
64         load_info.token_id      // 访问令牌, 用于权限校验
65     );
66     if (local_vaddr == MAP_FAILED) return -1;
67
68     // 更新全局指针, 供SVE计算使用
69     g_embedding_table_ub = (float*)local_vaddr;
70     g_embedding_token_id = load_info.token_id;
71
72     return 0;
73 }

```

真正的 UVB over UB Message 交互和 UB内存事务 的细节, 都由OS UVB Driver和灵衢固件在底层处理。

SVE核心计算逻辑

Code block

```

1  // 3. SVE核心计算逻辑: 对应用层完全透明
2  void sve_batch_lookup_on_ubmem(
3      const uint64_t* __restrict__ raw_features,
4      float* __restrict__ results,
5      size_t batch_size
6  ) {
7      svbool_t pg = svptrue_b64();
8      size_t vl = svcntd(); // 向量长度
9
10     for (size_t h = 0; h < batch_size; h += vl) {
11         svbool_t pg_loop = svwhilelt_b64(h, batch_size);
12         if (!svptest_any(pg_loop, pg_loop)) break;
13
14         // 流式预取, 避免污染L3缓存
15         svprfb(pg_loop, &raw_features[h], SV_PLDL1STRM);
16
17         // 加载原始特征ID
18         svuint64_t raw_v = svld1_u64(pg_loop, &raw_features[h]);
19         svuint64_t feat_id = svand_x(pg_loop, raw_v, svdup_u64(0xFFFFFFFFFFFFFFFFULL));
20
21         // 【关键】从OS映射的虚拟地址中Gather Load Embedding
22         // g_embedding_table_ub 是一个普通的本地虚拟地址
23         svfloat32_t emb = svld1_gather_u64index_f32(pg_loop, g_embedding_table_ub, feat_id);
24
25         // ... (后续加权、存储等逻辑保持不变) ...
26         svstnt1_f32(pg_loop, &results[h], emb);
27     }
28 }

```


在UB超节点架构下，**内核态的核心作用并非处理网络数据包，而是作为“资源管理者”和“安全仲裁者”**，为用户态应用提供安全、高效、标准化的底层硬件访问能力。

对应的标准化接口说明

以上伪代码依赖于两个核心接口，它们是对固件规范的封装：

接口	功能	底层实现
<code>ubios_call(CallID, ReceiverID, input, output)</code>	封装标准的 <code>Call ID Service</code> 调用。它会构造符合规范的 <code>UVB over UB Message</code> ，通过OS UVB Driver发送给目标UBPU的固件。	对应固件规范第7.3.3节定义三个核心接口： <code>0xC00B0020</code>：镜像服务申请 <code>0xC00B0021</code>：镜像服务能力查询 <code>0xC00B0022</code>：镜像信息获取
<code>ubios_mmap_remote(ubas_addr, size, token_id)</code>	请求OS内核将指定的远端UBAS地址空间，通过MMIO或类似机制，映射到当前进程的虚拟地址空间。	对应固件规范第6.2节 <code>MMIO映射</code> 。OS内核会配置本地UB Decoder页表，并利用 <code>token_id</code> 向远端UMMU申请访问权限。

8.1 内核旁路（Kernel Bypass）最小闭环

完全可以实现“仅通过用户态来完成网络请求和响应”，但这需要内核态提供一个极其精简但至关重要的支撑。

8.1.1 内核态的核心职责

内核态不参与数据面（Data Plane）的处理，其核心职责集中在控制面（Control Plane）和资源管理：

1. 硬件资源发现与初始化 (Resource Discovery & Initialization)

- 做什么：** 在系统启动时，内核驱动（即UBIOS组件）从固件（Firmware）处读取并解析 `UBC信息表`、`UMMU信息表`、`UB Entity信息表` 等。
- 为什么重要：** 这是用户态程序能够工作的前提。没有内核的初始化，用户态程序根本不知道远端内存的地址、可用的Call ID列表、以及如何与固件交互。

2. 虚拟内存映射 (Virtual Memory Mapping)

- 做什么：** 内核驱动将固件上报的 `UB Reserved Memory`（即远端共享内存池）的物理地址（或I/O地址）**映射到用户进程的虚拟地址空间**。
- 为什么重要：** 这是实现“同步内存语义”的关键。用户态SVE代码可以直接使用 `load / store` 指令访问这块内存，就像访问本地内存一样，完全绕过了内核的数据拷贝。

3. 安全与权限控制 (Security & Permission Control)

- 做什么：** 内核驱动负责验证用户态程序的请求是否合法。例如，当用户态调用 `uvb_call()` 时，内核会检查该进程是否有权访问目标 `Call ID` 和对应的内存区域。
- 为什么重要：** 防止恶意或错误的用户程序破坏系统稳定性或访问未授权资源。

4. 事件通知通道建立 (Event Channel Setup)

- 做什么：** 内核为用户态程序创建一个高效的、无锁的事件通知机制（如Ring Buffer或UIO设备），用于接收来自固件的异步通知（如 `Notify ID Information`）。
- 为什么重要：** 用户态程序要一种低开销的方式知道何时有新消息到达或操作完成，而无需轮询或陷入内核。

8.2 用户态如何完成网络请求与响应（关键环节）

在内核完成了上述“一次性”或“低频”的初始化工作后，**高频的数据交互完全在用户态完成**。具体流程如下：

场景：SVE引擎向远端请求加载一个新的Embedding分片

1. 用户态发起请求 (User-space Request)

- SVE应用调用一个**用户态库函数**（如 `ub_load_shard("shard_001")`）。
- 该库函数内部：
 - a. **构造标准请求包**：根据固件规范，填充一个符合 `Call ID: 0xC00B0022`（镜像信息获取）格式的请求结构体。
 - b. **直接写入共享内存**：将这个请求包**直接写入**由内核预先映射好的、与本地UBPU固件通信的**共享内存队列**（Submission Queue, SQ）中。**此过程不经过内核！**

2. 固件处理与转发 (Firmware Processing)

- 本地UBPU的灵衢固件持续监控SQ。
- 发现新请求后，固件将其封装成 `UVB over UB Message`，并通过UB总线发送给目标UBPU（通常是UBFM或存储服务节点）。
- **整个过程在固件和硬件层面完成，内核完全不参与。**

3. 远端响应与数据准备 (Remote Response & Data Prep)

- 目标UBPU的固件收到消息，处理请求（如查找文件元数据），并将响应（包含文件在UBAS中的地址和Token）以及实际的Embedding数据准备好。
- 响应通过UB总线发回。

4. 用户态接收响应 (User-space Response Handling)

- **方式一（主动查询）❌**：SVE应用可以定期检查由内核映射的**完成队列**（Completion Queue, CQ）的头指针（位于共享内存中）。一旦发现新条目，就读取响应。**无内核介入。**
- **方式二（事件驱动 - 推荐）✅**：
 - a. 固件在将响应写入CQ的同时，会向之前由内核设置好的**事件通知通道**（如Ring Buffer）写入一个事件。
 - b. SVE应用通过一个轻量级的**轮询线程**（Polling Thread）监听这个Ring Buffer。
 - c. 一旦发现事件，立即处理CQ中的响应。**整个过程依然是用户态到用户态，延迟极低。**

5. 用户态直接访问数据 (Direct Data Access)

- 响应中包含了Embedding数据在全球UBAS中的地址。
- 由于内核在初始化时已经将整个 `UB Reserved Memory` 区域映射到了SVE进程的地址空间，SVE代码现在可以**直接使用SVE向量指令**（如 `svld1_gather`）从这个虚拟地址加载数据进行计算。
- **这是真正的零拷贝、无中断、无内核开销的内存访问。**

8.3 与传统TCP/IP栈的对比

环节	传统TCP/IP栈	UB超节点 + 内核旁路
请求发起	<code>send()</code> 系统调用 → 陷入内核	直接写入共享内存SQ → 无内核
协议处理	内核协议栈（TCP/IP）处理	固件处理（UVB over UB Message）→ 无内核
数据接收	<code>recv()</code> 系统调用 → 陷入内核，数据从内核缓冲区拷贝到用户缓冲区	轮询CQ或事件通道 → 无内核 ；数据已在用户态可访问的内存中
内存访问	应用访问自己的缓冲区	应用直接访问全局共享内存（UB.MEM）

9. 风险与应对更新

风险	更新后的应对措施
URMA生态封闭	转向标准UVB接口 ：鲲鹏是否有《UB-Firmware-Specification-2.0-zh.pdf》的 OS UVB Driver 以及配套的 用户态开发SDK ，该SDK应封装标准的 <code>Call ID</code> <code>Service</code> 调用。
调试困难	利用标准RAS机制 ：固件规范第6.4节定义了标准的 <code>RAS信息上报接口</code> （Notify ID: <code>0x400B0200</code> ）。应要求提供基于此标准的 <code>UB Trace Analyzer</code> 工具。
与现有网关不兼容	在OS层做协议转换 ：在 <code>service_gateway</code> 层，将gRPC/HTTP请求转换为对本地OS UVB Driver的调用，由Driver负责跨UBPU通信。

小结

在UB超节点架构下，**内核态的核心是“赋能者”而非“执行者”**。它通过一次性的资源映射和安全设置，为用户态应用打开了通往硬件的“高速公路”。之后，所有高频的“网络”请求（实质上是远程过程调用RPC和内存访问）都由用户态应用通过**直接读写共享内存**和**监听轻量级事件**来完成，从而实现了极致的性能和效率。这将CPU从繁重的网络协议处理中解放出来，专注于核心的SVE计算任务。